

```

Terminal
satya@problem2$ ./a.out
The entire hex value is 0x1A1B1C1D
The first byte is 0x1D @ 0xbfe7c478
The second byte is 0x1C @ 0xbfe7c479
The third byte is 0x1B @ 0xbfe7c47a
The fourth byte is 0x1A @ 0xbfe7c47b
satya@problem2$

```

Figure 4: Output of the code `byte_order.c`

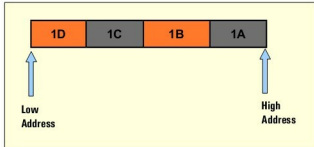


Figure 5: Byte-ordering in a Little-Endian machine

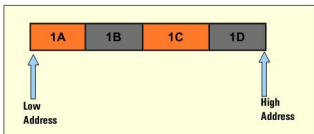


Figure 6: Byte-ordering in a Big-Endian machine

When I run this code in my system, I get the output shown in Figure 4.

From Figure 4, it is very clear that the underlying architecture is following the little Endian. When the same code is run under a different architecture, which follows Big Endian, the result will be different. So, portability issues need to be considered while using bit fields.

Let's look at one more example to understand how bits are packed in Big Endian and Little Endian.

To start with, let us consider the sample code (Labelled as `bit_order.c`) given below:

```

1 #include <stdio.h>
2 typedef union {
3     unsigned short value;
4     struct {
5         unsigned short v1 : 1;
6         unsigned short v2 : 2;
7         unsigned short v3 : 3;
8         unsigned short v4 : 4;
9         unsigned short v5 : 5;
10    } bit;
11 } data_t;
12
13 int main() {
14

```

```

15     data_t var ;
16     unsigned char *ptr = (unsigned char*)&var;
17     var.bit.v1 = 1;
18     var.bit.v2 = 2;
19     var.bit.v3 = 3;
20     var.bit.v4 = 4;
21     var.bit.v5 = 5;
22
23     printf("The Entire hex value is 0x%X\n", var.
value);
24     printf("The first byte is 0x%X @ %p\n", *(ptr +
0), ptr + 0);
25     printf("The second byte is 0x%X @ %p\n", *(ptr +
1), ptr + 1);
26
27     return 0;
28 }

```

```

Terminal
satya@problem2$ ./a.out
The Entire hex value is 0x151D
The first byte is 0x1D @ 0xbfe909da
The second byte is 0x15 @ 0xbfe909db
satya@problem2$

```

Figure 7: Output of the code `bit_order.c`

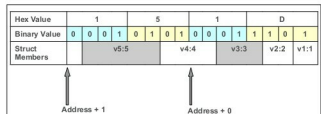


Figure 8: Bit ordering in small Endian architecture

When I run this code in my system, I get the output as shown in Figure 7.

From this figure, one can see that the bits are packed from the least significant on a little Endian machine. Figure 8 helps us understand how the bits ordering takes place.

If you run the same code in big Endian architecture, you will get the output given in Figure 9.

For more clarity, see Figure 10.

From the last two examples, it is very clear that bit fields pose serious portability issues. When the same programs are compiled on *different* systems, they may not *work* properly. This is because some C compilers use the left-to-right order, while other C compilers use the right-to-left order. They also have architecture-specific bit orders and packing issues.

As a concluding note, let us list the advantages and limitations of bit fields structures.

## Advantages

1. Efficiency - Storage of data structures by packing.